Version : 12 mars 2025

Master 1 GCE - Année scolaire 2024-2025

C.DESODT R. LECLERCQ

MSG22 - Machine Learning

Apprentissage supervisé - Partie 4 *(Supervised Learning)*

Résumé : Dans cette seconde partie du module, vous allez découvrir certaines méthodes d'apprentissage supervisé.

Mots clés :
FR : apprentissage supervisé, réseau de neurones, perceptron, convolution
ENG : supervised learning, neural network, perceptron, convolution

---

Problématique : l'objectif est de créer un réseau de neurones convolutif permettant de détecter la présence de fissures dans des images de murs en maçonnerie (cf. Figure 1).



FIGURE 1: Fissurations de maçonnerie

1. Données d'entrée

Les données utilisées dans ce BE sont récupérées du travail présenté dans [Hal+21].

- Télécharger le notebook "Maconnerie_Debut.ipynb" sur e-campus.
- Enregistrer le dossier "images" disponible sur e-campus dans le répertoire de votre notebook.
- Vérifier que 856 images ont été chargées en compilant le bloc d'import des images.
- Créer un jeu de données appelé `dataset` à partir de la classe `Dataset`. La classe `Dataset` prend deux variables en entrée : x et y qui contiennent respectivement les images et les étiquettes (labels) de ces images.
- Réaliser une séparation des données d'entraînement - 600 images -, de validation - 128 images - et de test - 128 images - (utiliser la méthode `random_split` de `torch.utils.data`)

- Visualiser des images de maçonnerie avec et sans fissure. Chaque image est stockée en *array* dans `dataset.data` (utiliser la méthode `imshow` de matplotlib).
- Rappel sur le découpage des données Dans les projets d'apprentissage automatique, nous devons généralement envisager plusieurs modèles avant de choisir le dernier qui sera finalement utilisé pour effectuer des prédictions sur des données du monde réel. Pour pouvoir entraîner les modèles, effectuer la sélection des modèles et enfin évaluer le modèle final afin de vérifier s'il peut bien se généraliser, nous divisons généralement l'ensemble de données original en ensembles d'entraînement, de test et de validation (cf figure 2).
- Train set : L'ensemble d'apprentissage est généralement le plus grand - en termes de taille - ensemble qui est créé à partir de l'ensemble de données original et qui est utilisé pour fidéliser le modèle.
- Validation set : L'ensemble de données de validation est maintenant utile lorsqu'il s'agit de régler les hyperparamètres et de sélectionner le modèle. Les exemples de validation inclus dans cet ensemble seront utilisés pour trouver les valeurs optimales des hyper-paramètres du modèle considéré.
- Test set : L'ensemble de test est utilisé pour évaluer les performances de ce modèle et s'assurer qu'il peut être généralisé à de nouveaux points de données non vus.

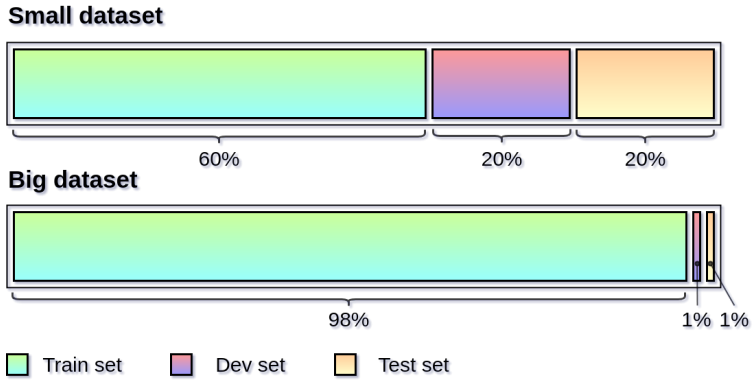


FIGURE 2: Découpage des données. Source : `https://machinelearning101.readthedocs.io/en/latest/_pages/07_model_performance.html`.

2. Perceptron multi-couches

Vous avez programmé votre perceptron multi-couches (cf Figure 3) au cours du BE précédent mais afin de bénéficier de fonctionnalités supplémentaires, nous utiliserons la libraire pyTorch pour faire de l'apprentissage machine.

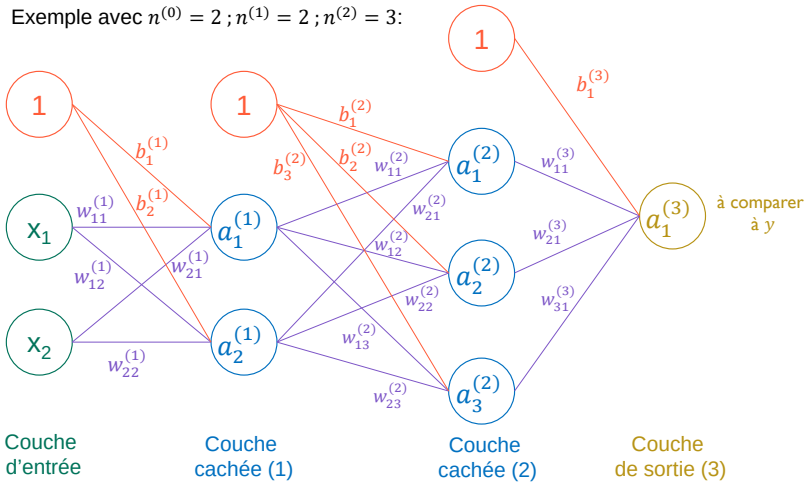


FIGURE 3: Perceptron multi-couche

La librairie `torch.nn` fournit tous les blocs de construction dont vous avez besoin pour créer votre propre réseau de neurones : couches entièrement connectées, couches convolutionnelles, etc.

2.1. Création et entraînement du modèle

- Régler les paramètres globaux (hyperparamètres) de votre modèle de la façon suivante :
  - Nombre d'époques [1] $= 20$
  - Taille des batchs [2] $= 10$
  - Fonction coût, ici calculée à partir des moindres carrés (utiliser la classe `MSELoss` de torch)
  - Algorithme d'optimisation, ici par gradient stochastique. Le taux d'apprentissage `lr` sera fixé à $0,01$ (utiliser la classe `SGD` de torch)
- Créer le modèle de perceptron multi-couches (utiliser la classe `Modele_MLP` fournie dans le notebook sans indiquer d'argument à l'appel de la classe)
- Utiliser la fonction d'entraînement `train` fournie dans le notebook.

1. une époque *(epoch)* correspond à un apprentissage sur toutes les données. Augmenter ce nombre permet de gagner en précision sur la base d'entrainement mais peu entrainer un sur-apprentissage *(overfitting)*.

2. un *batch* correspond à un lot de données. Si la taille de ce lot correspond au nombre d'images alors les gradients seront calculés sur l'ensemble des données en une fois. Sinon le calcul se fera par gradient stochastique.

2.2. Exploitation

- Visualiser l'évolution de l'erreur sur les datasets d'entraînement et de validation en fonction du nombre d'époques *(epochs)* (avec les paramètres ci-dessus). Que remarquez-vous ?
- Utiliser la fonction d'évaluation pour déterminer la précision (*precision*), le rappel (*recall*) et le F1-score (cf. Figure 4). Comparer les résultats obtenus entre les jeux de données d'entraînement et de test (utiliser la fonction `evaluation` fournie dans le notebook).

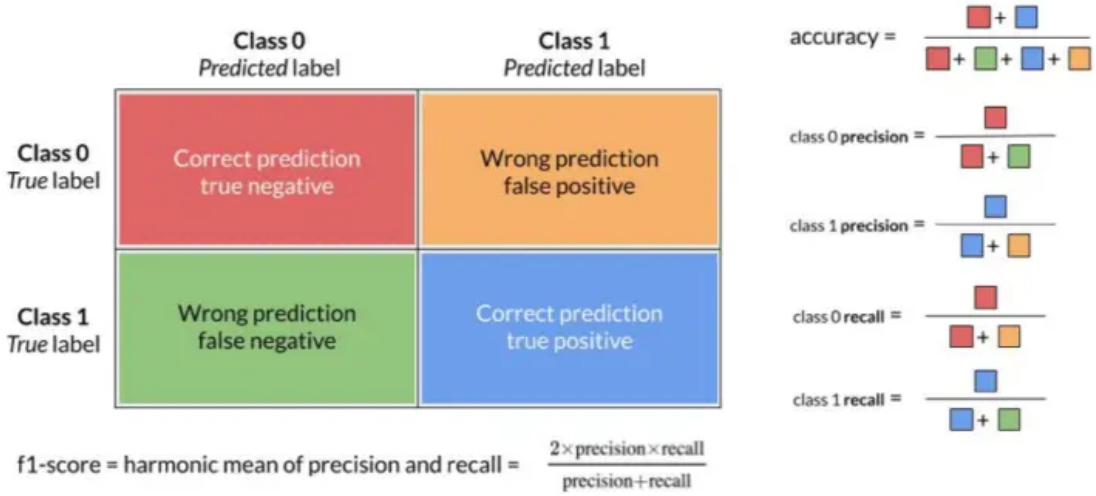


FIGURE 4: Évaluations (source = datascientest.com)

- (Facultatif) Montrer l'impact du nombre total d'époques *(epochs)* et du taux d'apprentissage sur l'erreur calculée.

3. Réseau de neurones convolutif

Le modèle étudié par la suite ne sera plus 'linéaire" mais "convolutif". Les réseaux de neurones convolutifs ont une méthodologie similaire à celle étudiée précédemment : ils reçoivent des images en entrée, détectent les caractéristiques de chacune d'entre elles, puis entraînent un classifieur (cf Figure 5).

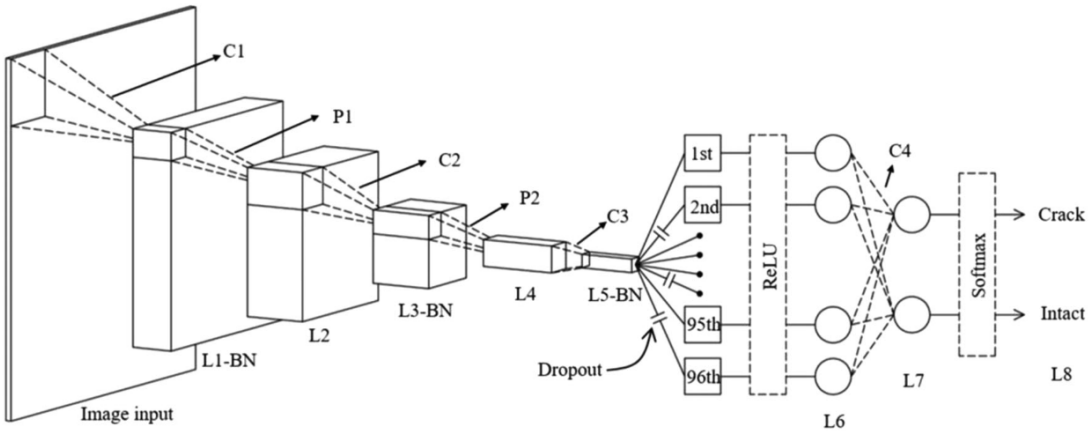


FIGURE 5: Réseau de neurones convolutif (source = [CC17])

Les réseaux de neurones convolutifs, aussi appelés CNN ou ConvNet pour Convolutional Neural Network, sont toujours les modèles les plus performants pour la classification d'images (source : openclassrooms.com). Leur architecture est composée de deux blocs principaux :

- Le premier bloc fait la particularité de ce type de réseaux de neurones, puisqu'il fonctionne comme un extracteur de caractéristiques (ou *features*). Pour cela, il effectue du *template matching* en appliquant des opérations de filtrage par convolution. La première couche filtre l'image avec plusieurs noyaux de convolution, et renvoie des "feature maps", qui sont ensuite normalisées (avec une fonction d'activation) et/ou redimensionnées. Ce procédé peut être réitéré plusieurs fois : on filtre les *features maps* obtenues avec de nouveaux noyaux (filtres), ce qui nous donne de nouvelles *features maps* à normaliser et redimensionner, et qu'on peut filtrer à nouveau, et ainsi de suite. Finalement, les valeurs des dernières *feature maps* sont concaténées dans un vecteur. Ce vecteur définit la sortie du premier bloc, et l'entrée du second.
- Le second bloc n'est pas caractéristique d'un CNN : il se retrouve en fait à la fin de tous les réseaux de neurones utilisés pour la classification. Les valeurs du vecteur en entrée sont transformées (avec plusieurs combinaisons linéaires et fonctions d'activation) pour renvoyer un nouveau vecteur en sortie. Ce dernier vecteur contient autant d'éléments qu'il y a de classes : l'élément i représente la probabilité que l'image appartienne à la classe i. Chaque élément est donc compris entre 0 et 1, et la somme de tous vaut 1. Ces probabilités sont calculées par la dernière couche de ce bloc (et donc

du réseau), qui utilise une fonction logistique (classification binaire) ou une fonction softmax (classification multi-classe) comme fonction d'activation.

Un avantage de ce type de réseau est l'utilisation d'un poids unique associé aux signaux entrant dans tous les neurones d'un même filtre de convolution. Cela réduit la quantité de mémoire allouée par l'algorithme, améliore les performances (cf Figure 6). C'est le principal avantage du réseau de neurones convolutifs par rapport au perceptron multicouche, qui, lui, considère chaque neurone indépendant et affecte donc un poids différent à chaque signal entrant.

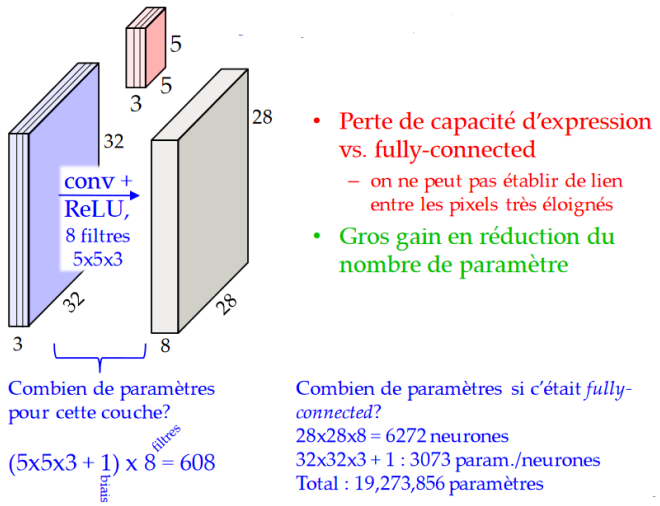


FIGURE 6: Comparaison du nombre de paramètres entre une étape de convolution et celle d'activation classique (*fully connected*) (source = `http://cs231n.stanford.edu/`)

De nombreuses explications et détails sur les différents filtres sont décrits sur le site stanford.edu. Les différentes étapes de traitement d'image amènent à compresser les données. La représentation de ces données compressées sont de plus en plus abstraites (cf Figure 7).

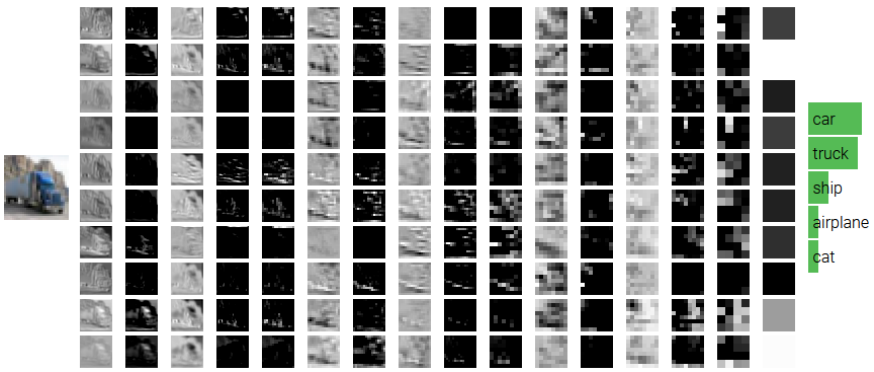


FIGURE 7: Étapes de traitement d'image par CNN (source = `http://cs231n.stanford.edu/`)

3.1. Modèle convolutif

- Régler les paramètres globaux de votre modèle de la façon suivante :
  - Nombre d'epochs = 20
  - Taille des batchs = 10
  - Fonction coût, ici calculée à partir des moindres carrés (utiliser la classe `MSELoss` de torch)
  - Algorithme d'optimisation, ici par gradient stochastique. Le taux d'apprentissage `lr` sera fixé à $0,01$ (utiliser la classe `SGD` de torch)
- Créer le modèle convolutif (utiliser la classe `Modele_ConvNet_1` fournie dans le notebook sans indiquer d'argument à l'appel de la classe). Ce modèle se décrit comme suit :
  - Le filtre de la couche de convolution correspond à un carré de côté = 3 pixels (largeur moyenne d'une fissure). Le nombre de cartes de convolution créées est arbitrairement de 16.
  - Une étape de *DropOut* retire 25% des connections pour éviter le surapprentissage.
  - Une fonction d'activation RELU est ensuite utilisée (cf Figure 8). Cette fonction permet d'effectuer un filtre sur nos données. Elle laisse passer les valeurs positives dans les couches suivantes du réseau de neurones. Elle est utilisée parfois dans les couches intermédiaires uniquement.

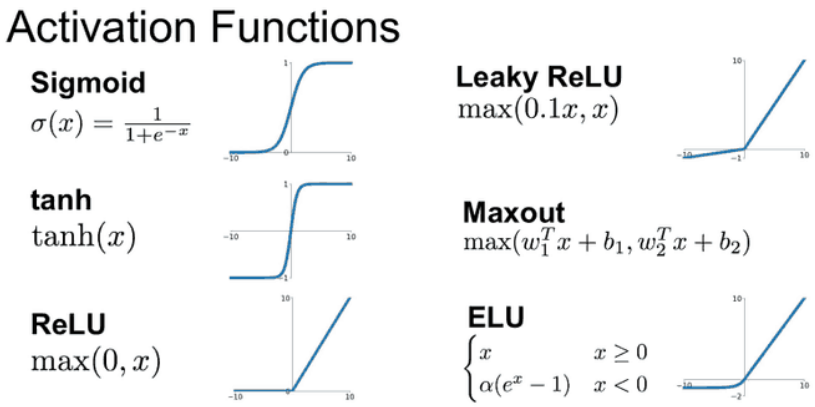


FIGURE 8: Fonctions d'activations (source = moncoachdata.com)

  - Les données sont vectorisées par la méthode `flatten`.
  - La dernière couche intermédiaire correspond à une couche d'activation classique.
- Utiliser la fonction d'entraînement `train` fournie dans le notebook.
- Comparer les résultats entre les deux types de réseaux de neurones. Quel comportement remarquez-vous concernant l'erreur sur le dataset de validation en fonction des époques ?
- Combien d'époques sont en fait nécessaires pour entraîner ce réseau de neurones sans sur-apprentissage ?

3.2. Grille de recherche (GridSearch)

Les paramètres globaux de réseau de neurones convolutif sont nombreux. Pour évaluer la sensibilité de certains paramètres, et surtout déterminer un jeu optimal de paramètres, on réalise souvent une grille de recherche. On choisira le modèle `Modele_ConvNet_2` car il permet plusieurs couches convolutives.

- Modifier le dictionnaire `params` afin de réaliser la grille de recherche sur différents jeux de paramètres (10 max). Vous êtes libres sur les choix de paramètres.
- Appliquer la fonction `GridSearch`. Commenter.

3.3. Utilisation d'un réseau de neurones pré-entraîné

Les deux architectures A et B de l'article de [Hal+21] ont été testées en amont.

- Récupérer les fichiers "CNN_architecture_A_batch_size_20_epochs_200" et "CNN_architecture_B_batch_size_20_epochs_200" dans le dossier contenant le notebook.
- Importer successivement les deux modèles (utiliser la méthode `load_state_dict` de torch)
- Déterminer les matrices de confusion pour les jeux de données d'entraînement et de test. Comparer.

3.4. (Facultatif) Visualisation des filtres

La fonction `visualisation_parametres` permet de visualiser les paramètres d'un modèle convolutif et notamment les filtres des couches de convolution (NB : seule les 'faces' supérieures des couches ayant une troisième dimension sont représentées).

- Visualiser les filtres de l'architecture A et/ou B et commenter.

Références

[CC17] Young-Jin Cha et Wooram Choi. “Deep Learning-Based Crack Damage Detection Using Convolutional Neural Networks”. In : *Computer-Aided Civil and Infrastructure Engineering* (2017).

[Hal+21] Mitchell J Hallee et al. “Dataset for Crack Detection in Images of Masonry Using CNNs”. In : *Sensors* (2021). doi : 10.5281/zenodo.5108846.